How do I get that number?

Pick a number and follow it back to the bytes it came from. Every stage names the module and function that runs it, the library doing the work, and what the number actually looks like at that point.

engine v0.4.0 · contract v0.2.0-draft · 178 tests · profiles b2b_industrial, dairy_al_kharj

---

The interactive version of this document shows one trace at a time; this one shows all three, with the arithmetic already open. Part one describes the pipeline once. Part two follows three different numbers through it - including one that deliberately refuses to answer.

Part one · The pipeline

Ten stages, in the order they run.

1. Source

`data/ · axiom-source-mapping.json`

DVC · openpyxl · pandas

The bytes the answer is made of, pinned so the same run can be repeated.

2. Load & type

`graph/source.py · mapping/workbook.py`

pandas · openpyxl

Named tables in, typed frames out. The one interface the engine is allowed to depend on.

3. Admission

`quality/conformance.py · graph/validation.py · quality/readiness.py`

pandas

Does this data fit, is it sound, and what can it actually support? Answered before anything is computed.

4. Extract

`profiles.py · _b2b_extract / _dairy_extract`

pandas

The profile's own reading of the source: line-grain measures, document-grain measures, dimensions.

5. Walk the chain

`engine/waterfall.py · apply_chain, _roll_up`

pandas · numpy

The declared step chain, computed at the grain each measure is actually known at.

6. Certify

`engine/waterfall.py · _certify`

pandas

Which documents are fully decomposable, and the sentence explaining each that is not.

7. Aggregate

`engine/aggregation.py · aggregate`

`pandas`

Roll up to any dimension the grain supports — and abstain, with a reason, where it does not.

8. Contract

`engine/result.py · contracts/schema.py`

`dataclasses · json`

The typed object Track B receives, re-checked against its own arithmetic before it is emitted.

9. Models

`models/ · segmentation, elasticity, acceptance, optimiser`

`scikit-learn · PyMC · ArviZ · numpy`

D4–D6. Fitted only where the readiness score says the data can carry them.

10. Track

`tracking/mlflow_runs.py · dvc.yaml`

`MLflow · DVC`

The run, keyed to the hash of the data that produced it.

Trace · Pocket margin, whole book

Answer: 4,119,830.86 EUR · b2b_industrial · CSV snapshot

1. Source

CARRIES THE NUMBER | 8 CSVs · 4,188 rows

IN	OUT
data/snapshot_synthetic_v1/ transactions.csv 700 transaction_lines.csv 1,408 products.csv 24 + 5 more	bytes on disk, pinned by DVC

The snapshot is an input the pipeline receives, not one it produces, so DVC pins its bytes rather than rebuilding it. The sha256 over these files travels all the way to the result object.

2. Load & type

CARRIES THE NUMBER | typed frames

IN	OUT
transaction_lines.csv	DataFrame(1408 x 5) quantity int64 line_total float64

A CSV round-trip makes every column text. applies the profile's declared semantic types, so is a number before anything compares it. Unparseable values become NaN, which the engine already treats as UNAVAILABLE rather than as zero.

3. Admission

CARRIES THE NUMBER | EXACT · 83.3/100 · no blockers

IN	OUT
the 8 loaded tables	conformance EXACT readiness 83.3 READY 0 blocking findings

Three gates, cheapest first. Conformance asks whether the files match the profile at all; validation checks referential integrity and semantic types; readiness scores what the data can support. A dangling product reference stops the run here rather than producing a smaller book that looks fine.

4. Extract

CARRIES THE NUMBER | **1,408 line rows + 700 doc rows**

IN	OUT
transaction_lines + products + transactions + customers/regions/channels	Extract(lines = 1,408 rows keys = 700 rows)

One join, once, up front. The line-grain measures are computed here; the order's two discount edges are read separately and deliberately left at order grain.

row	measure	value
TRX-R04 / PRD-1	quantity 10 x list_price 100	1,000.00
TRX-R04 / PRD-1	line_total as invoiced	800.00
TRX-R04 / PRD-1	quantity 10 x unit_cost 60	600.00
TRX-R04 / PRD-3	quantity 10 x list_price 50	500.00
TRX-R04 / PRD-3	line_total as invoiced	400.00
TRX-R04 / PRD-3	quantity 10 x unit_cost 20	200.00

5. Walk the chain

CARRIES THE NUMBER | **TRX-R04 → 208.00**

IN	OUT
Extract(lines, keys) + spec.Profile chain	facts (line grain, 4 steps) keys (doc grain, 9 steps)

The chain is walked twice: once at line grain for the steps knowable per line, then the line measures are summed *up* into the order and the rest of the chain runs there. Summing up needs no assumption. Dividing down would.

step	how	TRX-R04
List value	1,000 + 500	1,500.00
- negotiated off-list	1,500 - 1,200	300.00
= Gross value	800 + 400	1,200.00
- on-invoice discount	DSC-ON-A, at order grain	120.00
= Invoice price	1,200 - 120	1,080.00
- off-invoice leakage	DSC-OFF-A 48 + DSC-OFF-B 24	72.00
= Pocket price	1,080 - 72	1,008.00
- COGS	600 + 200	800.00
= Pocket margin	1,008 - 800	**208.00**

6. Certify

CARRIES THE NUMBER | **530 of 700 certified**

IN	OUT
keys (700 rows)	certified = True 530 certified = False 170 + uncertified_reason

A document is certified when every measure the chain needs is present and no discount edge has an unrecorded amount. 170 orders fail — 113 for a product with no unit cost, 60 for no list price, 11 for a discount whose amount nobody wrote down. Each carries the sentence that says which.

7. Aggregate

CARRIES THE NUMBER | **4,119,830.86**

IN	OUT
keys, certified only	one row per group + coverage_pct + grain_note

Coverage is 75.71% — 530 of 700 orders — and it is printed beside the total rather than behind it. A step is summed only when every contributing row has it, so a partial sum can never masquerade as a result.

step	how	total
List value	sum over 530 certified orders	31,108,764.18
- negotiated off-list		3,688,235.18
= Gross value		27,420,529.00
- on-invoice discount		3,669,609.59
= Invoice price		23,750,919.41
- off-invoice leakage		1,117,996.07
= Pocket price		22,632,923.34
- COGS		18,513,092.48
= Pocket margin		**4,119,830.86**

8. Contract

CARRIES THE NUMBER | **Money(4119830.86, "EUR")**

IN	OUT
aggregated rows	EngineResult(waterfalls=(...), provenance=...,)

re-derives the arithmetic it can check — invoice price must equal gross minus on-invoice, pocket must equal invoice minus leakage — and rejects the payload if it does not reconcile. It also refuses any COMPLETE record that cites no evidence.

9. Models

NOT ON THIS PATH | **not on this path**

The headline waterfall does not go through the model layer. Segmentation, elasticity and price recommendation read the same fact tables but produce different records.

10. Track

CARRIES THE NUMBER | **run logged, snapshot hash recorded**

IN	OUT
totals, flags, availability, the result JSON	MLflow run + reports/*.csv

MLflow records the run; DVC records the bytes. They meet at the snapshot hash, so given a run you can recover the exact data, and given the data you can regenerate the run.

Trace · On-invoice discount, by product

Answer: UNAVAILABLE — and why · b2b_industrial · the grain rule

1. Source

CARRIES THE NUMBER | transaction_discounts.csv · 1,997 rows

IN	OUT
transaction_discounts.csv transaction_id discount_id discount_amount_applied	1,997 edges over 700 orders

Look closely at the columns. The edge carries a transaction, a discount and an amount. There is no product on it. That single absence is the whole of this trace.

2. Load & type

CARRIES THE NUMBER | typed

IN	OUT
transaction_discounts.csv	discount_amount_applied float64

An amount that is blank stays NaN rather than becoming zero. The engine keeps three states apart: no edge at all (a true zero), an edge with no amount (unknown), and a field never populated (a gap).

3. Admission

CARRIES THE NUMBER | warning raised

IN	OUT
the discount edges	WARNING discount amounts: 15 applied with unknown amount

Validation flags this as a warning, not a blocker — it degrades a result without invalidating it, and it travels into the confidence reasons on the contract record.

4. Extract

CARRIES THE NUMBER | kept at order grain

IN	OUT
transaction_discounts + discounts (for the type)	keys frame: on_invoice_discount off_invoice_leakage n_unknown_*

This is the function where the pro-rata allocation used to live. It returned the discount against each *line*, split in proportion to line value. It now returns it against the order and stops.

order	edges	amount
TRX-R04	on-invoice edge DSC-ON-A	120.00
TRX-R04	off-invoice DSC-OFF-A + DSC-OFF-B	72.00

5. Walk the chain

CARRIES THE NUMBER | **grain propagates down the chain**

IN	OUT
`Step(on_invoice_discount, grain=KEY)`	`every step below it: grain=KEY`

computes this once from the chain. A level built from a line-grain level and an order-grain deduction is knowable only at order grain, and so is everything below it — including pocket margin, even though COGS is a line-grain measure.

step	origin	grain
list_value	measure	LINE
negotiated_off_list	derived	LINE
gross_value	measure	LINE
on_invoice_discount	measure	KEY
invoice_price	derived	KEY
off_invoice_leakage	measure	KEY
pocket_price	derived	KEY
cogs	measure	LINE
pocket_margin	derived	KEY

6. Certify

CARRIES THE NUMBER | **unchanged**

IN	OUT
`keys`	`certified flag`

Certification is unaffected: it asks whether the inputs are present, not where they can be reported.

7. Aggregate

ABSTAINS HERE | **NaN + grain_note**

IN	OUT

`aggregate(w, "product")`	`product_id on_invoice_discount`
	`PRD-001 NaN`
	`PRD-002 NaN`

is not constant inside an order, so the groupby reads the *line* frame — where the order-grain columns do not exist at all. A column that exists is a column someone will sum, so they are not there to be summed.

step	grain at product slice	PRD-1
list_value	LINE — survives the slice	2,200.00
negotiated_off_list	LINE	500.00
gross_value	LINE	1,700.00
cogs	LINE	1,320.00
on_invoice_discount	KEY — abstains	n/a
invoice_price	KEY — abstains	n/a
off_invoice_leakage	KEY — abstains	n/a
pocket_price	KEY — abstains	n/a
pocket_margin	KEY — abstains	n/a

8. Contract

ABSTAINS HERE | **Money.unavailable(reason)**

IN	OUT
the suppressed rows	{"amount": null, "status": "unavailable", "note": "...allocating it..."}

The reason travels with the null. Track B can write the sentence "we cannot attribute this discount to a product" instead of inferring something from a blank.

9. Models

NOT ON THIS PATH | **—**

Not on this path.

10. Track

CARRIES THE NUMBER | **availability logged per step**

IN	OUT
step availability	available_pct__on_invoice_discount

Per-step availability is logged on every run, so a drop in what the book can report shows up as a metric rather than as a surprise in a meeting.

Trace · Contribution margin, SKU-003

Answer: 218,356.37 SAR · dairy_al_kharj · mapped workbook

1. Source

CARRIES THE NUMBER | 1 mapping + 1 workbook

IN	OUT
axiom-source-mapping.json 86 triplesMaps 40 classes / 26 sheets Integrated_Dairy_Model...xlsx	the mapping is a schema, the workbook is the data

The mapping arrived before the workbook did, so runs the mapping backwards and generates a workbook with exactly the structure it declares. Every value in it is a placeholder; the sheets, columns and resolving keys are real.

2. Load & type

CARRIES THE NUMBER | sheet 24_Financial_Plan · 18 rows

IN	OUT
profile.tables["financial_plan"] -> "24_Financial_Plan"	DataFrame(18 x 12)

The profile asks for ; the mapping resolves that to sheet . That indirection is what lets one engine read a CSV snapshot and a client's workbook without knowing the difference.

3. Admission

CARRIES THE NUMBER | 80.3/100 · LIMITED

IN	OUT
13 mapped tables	waterfall READY segmentation BLOCKED elasticity BLOCKED optimisation BLOCKED

A good score cannot buy a READY. The mapping records no customer and no date on the plan sheet, so three of five deliverables are impossible here — and the score says so before anything is fitted rather than after something crashes.

4. Extract

CARRIES THE NUMBER | 18 plan rows

IN	OUT
24_Financial_Plan	one fact row per scenario x SKU x region

The plan row is the finest grain at which this mapping exposes any amount. maps InvoiceLine ids and links and not one measure; exposes and nothing else, so there is no list price anywhere.

measure	mapped from	SKU-003
gross_value	Planned_Revenue_SAR	657,213.73
off_invoice_leakage	Planned_Freight_SAR + Planned_Returns_SAR	38,365.60
cogs	Planned_COGS_SAR	400,491.76

5. Walk the chain

CARRIES THE NUMBER | **5 steps, all at row grain**

IN	OUT
the dairy Profile chain	gross_value - off_invoice_leakage = pocket_price - cogs = pocket_margin

A different chain, the same engine. Four canonical steps — list value, negotiated off-list, on-invoice discount, invoice price — are simply absent, because the mapping cannot support them. Because the key *is* the row, nothing ever abstains for grain here.

step	origin	SKU-003
Planned revenue	measure	657,213.73
- freight and returns	measure	38,365.60
= Net delivered revenue	derived	618,848.13
- product cost	measure	400,491.76
= Contribution margin	derived	**218,356.37**

6. Certify

CARRIES THE NUMBER | **18 of 18 certified**

IN	OUT
18 plan rows	coverage 100%

Every measure the chain needs is present on every row, so the whole plan is decomposable. Compare the CSV snapshot's 75.7%: a shorter chain is easier to satisfy, which is a fact about the chain rather than about the data quality.

7. Aggregate

CARRIES THE NUMBER | **218,356.37**

IN	OUT
aggregate(w, "product")	SKU-001 179,216.79 SKU-002 126,003.88 SKU-003 218,356.37

Every dimension the dairy profile declares is constant within a plan row, so the product slice reports all five steps. The engine also reconciles the result against the sheet's own — agreement to 2e-11.

8. Contract

CARRIES THE NUMBER | **Money(218356.37, "SAR")**

IN	OUT
the SKU rows	pocket_margin 218356.37 SAR list_value null + reason

The same Track B already reads. The steps this mapping cannot support come back as UNAVAILABLE carrying "no list price is mapped on any sheet; 08_Products exposes SKU_ID only" — a sentence someone can act on upstream.

9. Models

ABSTAINS HERE | **3 of 3 blocked**

IN	OUT
readiness capabilities	segmentation BLOCKED elasticity BLOCKED optimisation BLOCKED

Readiness is consulted first rather than caught afterwards. Running the fit and handling the exception would reach the same place, but it would reach it by accident and the reason would be a traceback rather than a sentence.

10. Track

CARRIES THE NUMBER | **hash covers mapping + workbook**

IN	OUT
workbook bytes + mapping bytes	content_sha256

Change the mapping and the same workbook is a different read, so the provenance hash covers both.

Reproduce any of this: dvc repro
Check your data first: python scripts/check_data.py
Provenance: every result carries a sha256 over the source bytes, so a number can always be traced to the exact snapshot that produced it.
Regenerate this document: python scripts/build_lineage.py